

University of Victoria

PWM Signal Generation and Monitoring System

Department of Electrical and Computer Engineering
University of Victoria
ECE 355 - Microprocessor-Based Systems

Connor Wiebe
Benjamin Philipenko

Lab Date: 21 November 2023
Submission Date: 28 November 2023

Table of Contents

1. Introduction....1
1.1 Problem Definition....1
1.1.1 Project Objectives....2
1.1.2 Key System Functionalities Implemented....2
2. Project Implementation & Design....3
2.1 Global Variable Overview....3
2.2 Initialization & Configuration of TIM2....3
2.3 Initialization & Configuration of GPIO Ports....4
2.3.1 GPIOA Initialization....4
2.3.2 GPIOB Initialization....5
2.3.3 GPIOC Initialization....6
2.4 Initialization & Configuration of External Interrupts....7
2.5 Analog-to-Digital Converter (ADC) Initialization....9
2.6 Digital-to-Analog Converter (DAC) Initialization....10
2.7 Initialization & Configuration of OLED Display....11
2.8 Interrupt Handling for EXTI0_1....16
2.9 Interrupt Handling for EXTI2_3....17
2.10 Interrupt Measurements for 555 Timer & Function Generator....17
2.11 Main Loop Execution....18
3. Testing & Results....20
3.1 Frequency Measurement Validation....20
3.2 Resistance Measurement Validation....23
3.3 ADC Voltage Validation & Potentiometer Integration....24
3.4 DAC Validation & Voltage Comparison....24
3.5 User Button Testing & Corrective Measures....25
3.6 OLED Display Testing for Frequency and Resistance Measurements....25
3.7 System Testing....27
4. Discussion....28
4.1 System Functionality and Objectives....28
4.2 Hardware Integration and Testing....28
4.3 System Robustness and Limitations....28
4.4 Recommendations for Future Enhancements....29
5. Conclusion....30
References....31
Appendix A: Source Code....32

1. Introduction

1.1 Problem Definition

In the ECE 355 Microprocessor-Based Systems course, the objective of this final lab project was to design and implement an embedded system for monitoring and controlling a Pulse Width Modulated (PWM) signal generated by a 555 timer (NE555 IC). The system utilizes an optocoupler (4N35 IC) driven by the STM32F051R8T6 microcontroller on the STM32F0 Discovery board to regulate the PWM signal's frequency and duty cycle. Additionally, the microcontroller is tasked with measuring the voltage across a potentiometer (POT) on the PBMCUSLK board and using the acquired data to influence the PWM signal frequency.

The lab manual also provided a comprehensive description of the signals and their associated pin information as shown in the table below.

Table 1. Specified pin assignments given by the lab manual

STM32F0	SIGNAL	DIRECTION
PA0	USER PUSH BUTTON	INPUT
PC8	BLUE LED	OUTPUT
PC9	GREEN LED	OUTPUT
PA1	555 TIMER	INPUT
PA2	FUNCTION GENERATOR	INPUT
PA4	DAC	OUTPUT (Analog)
PA5	ADC	INPUT (Analog)
PB3	SCLK (Display D0: “Serial Clock” = SPI SCK)	OUTPUT (AF0)
PB4	RES# (Display “Reset”)	OUTPUT
PB5	SDIN (Display D1: “Serial Data” = SPI MOSI)	OUTPUT (AF0)
PB6	CS# (Display “Chip Select”)	OUTPUT
PB7	D/C# (Display “Data/Command”)	OUTPUT

1.1.1 Project Objectives

I. Develop an interrupt-driven system that responds to the USER button press by toggling between measuring the frequency of the 555 timer signal and the Function Generator signal.

II. Employ the microcontroller's ADC in a polling approach to continuously measure the analog voltage signal from the potentiometer, allowing for the calculation of the potentiometer resistance.

III. Use the DAC to convert the digital value obtained from the ADC into an analog voltage signal, driving the optocoupler and adjusting the frequency and duty cycle of the 555 timer signal.

IV. Implement SPI communication to configure and drive the LED Display (SSD1306 IC) mounted on the PBMCUSLK board for displaying the measured signal frequency and potentiometer resistance.

V. Design an efficient and responsive system that achieves the specified requirements, ensuring accurate measurements and reliable control of the PWM signal.

1.1.2 Key System Functionalities Implemented

I. Continuous measurement of the analog voltage signal from the potentiometer using the microcontroller's internal Analog-to-Digital Converter (ADC) through a polling approach.

II. Calculation of the corresponding potentiometer resistance value based on the voltage measurements.

III. Utilization of the DAC (Digital-to-Analog Converter) to drive the optocoupler, adjusting the signal frequency and duty cycle of the 555 timer output in response to the potentiometer voltage.

IV. Measurement of the frequency of the 555 timer signal or the Function Generator signal using the microcontroller. The toggle between these measurements is triggered by the USER button, and an interrupt request is raised accordingly.

V. Displaying the measured signal frequency and the corresponding potentiometer resistance on the LED Display (SSD1306 IC) mounted on the PBMCUSLK board using SPI for communication.

2. Project Implementation & Design

In this section, we outline the source code and the design decisions behind it. We provide a thorough overview of the coding choices and the rationale behind the design to provide an understanding of the implementation.

2.1 Global Variable Overview

In this project, we've introduced several global variables to enhance functionality.

- *Freq* and *Res* are responsible for measuring and displaying frequency and resistance, respectively.
- *input_voltage* is implemented to accurately capture voltage from the ADC, which coincides with the *pot_measurement* variable determined by the *adc_dr* register.
- *pot_measurement* has two purposes, as it is used to store the input voltage from the ADC and also contributes to the calculation of potentiometer resistance.
- *inSig* is used to identify the origin of the detected frequency, differentiating signals originating from the 555 timer and those generated by the function generator.

Figure 1 showcases the initialization of these variables, providing a clear overview of their setup in the project.

```
// Initialize global variables
float Freq = 6969;  // measured frequency value (global variable)
unsigned int Res = 6969;   // measured resistance value (global variable)

float input_voltage = 0;
float dac_voltage = 0;
float pot_measurement = 0;

int inSig = 0;
```

Figure 1. Declaration of global variables implemented

2.2 Initialization & Configuration of TIM2

As can be seen in the initial line of Figure 2, the initialization process of TIM2 begins by enabling the clock. Subsequently, the TIM2 period is configured to a specified variable from lab 2, set to 0xFFFFFFFF. The TIM2 control register is then used to establish key parameters such as

buffer auto-reload, count-up, overflow stop, and the enabling of interrupt and update events. The TIM2 prescaler is deliberately set to 0x0 for this purpose. The EGR register is subsequently utilized to confirm the proper updating of the timer registers.

Finally, the NVIC priority is set, which is configured to assign TIM2 a priority of zero due to its critical role as the primary interrupt in the event of timer overflow. Following this, the DIER register is configured using a constant value, ensuring the timer's capability to generate interrupts is appropriately enabled.

```
void myTIM2_Init() {

    // Enable clock for TIM2 peripheral
    RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;

    // Configure TIM2: buffer auto-reload, count up, stop on overflow,
    // enable update events, interrupt on overflow only
    TIM2->ARR = myTIM2_PERIOD;
    TIM2->CR1 |= (uint16_t)0x8C;
    TIM2->CR1 &= (uint16_t)0xFF8C;

    // Set clock prescaler value
    TIM2->PSC = myTIM2_PRESCALER;

    // Update timer registers with above changes
    TIM2->EGR |= TIM_EGR_UG;

    // Assign TIM2 interrupt priority = 0 (highest) in NVIC
    NVIC_SetPriority(15, 0);

    // Enable TIM2 interrupts in NVIC
    NVIC_EnableIRQ(15);

    // Enable TIM2 interrupt generation
    TIM2->DIER |= TIM_DIER_UIE;

}
```

Figure 2. Initialization of Timer 2

2.3 Initialization & Configuration of GPIO Ports

2.3.1 GPIOA Initialization

To initiate the GPIOA configuration, the initial step involves enabling the clock, as indicated in the first line. Subsequently, PA2, PA4, and PA0 are designated as inputs for specific purposes:

PA2 for the 555 timer, PA4 for the ADC, and PA0 for the user button. Additionally, all pull-up/down resistors are deactivated for these ports.

```
void myGPIOA_Init() {

    /* Enable clock for GPIOA peripheral */
    RCC->AHBENR |= RCC_AHBENR_GPIOAEN;

    /* Configure PA2 as input */
    GPIOA->MODER &= ~(GPIO_MODER_MODER2);

    /* Ensure no pull-up/pull-down for PA2 */
    GPIOA->PUPDR &= ~(GPIO_PUPDR_PUPDR2);

    /* Configure PA4 as input */
    GPIOA->MODER |= GPIO_MODER_MODER4;

    /* Ensure no pull-up/pull-down for PA4 */
    GPIOA->PUPDR &= ~(GPIO_PUPDR_PUPDR4);


    // User Button Configuration:

    /* Configure PA0 as input */
    GPIOA->MODER &= ~(GPIO_MODER_MODER0);

    /* Ensure no pull-up/pull-down for PA0 */
    GPIOA->PUPDR &= ~(GPIO_PUPDR_PUPDR0);

}
```

Figure 3. Initialization of GPIOA port

2.3.2 GPIOB Initialization

To initiate the setup for GPIOB, the initial step mirrors that of GPIOA, involving the initialization of the clock. Subsequently, PB4, PB6, and PB7 undergo initialization similar to their equivalents in GPIOA, with the distinction that they are configured as outputs instead of inputs. Furthermore, PB3 and PB5 are configured to utilize their alternate functions, specifically serving as Serial Clock and Serial Data functions, respectively. Although the MODER register is employed for this configuration, the AFR register is later utilized to ensure that these pins are properly recognized by the board as utilizing alternate functions.

```c
void myGPIOB_Init() {

    /* Enable clock for GPIOB peripheral */
    RCC->AHBENR |= RCC_AHBENR_GPIOBEN;

    /* Configure PB3 to use alternate function SCK */
    GPIOB->MODER |= GPIO_MODER_MODER3_1;

    /* Ensure no pull-up/pull-down for PB3 */
    GPIOB->PUPDR &= ~(GPIO_PUPDR_PUPDR3);

    /* Configure PB4 as RES# (Reset) */
    GPIOB->MODER |= GPIO_MODER_MODER4_0;

    /* Ensure no pull-up/pull-down for PB4 */
    GPIOB->PUPDR &= ~(GPIO_PUPDR_PUPDR4);

    /* Configure PB5 as output MOSI (Serial Data) */
    GPIOB->MODER |= GPIO_MODER_MODER5_1;

    /* Ensure no pull-up/pull-down for PB5 */
    GPIOB->PUPDR &= ~(GPIO_PUPDR_PUPDR5);

    /* Configure board to recognize PB5 is being used as alternate function SCK (Serial Clock) */
    GPIOB->AFR[0] |= GPIO_AFRL_AFSEL0;

    /* Configure PB6 as CS# (Chip Select) */
    GPIOB->MODER |= GPIO_MODER_MODER6_0;

    /* Configure PB7 as D/C# (Data/Command) */
    GPIOB->MODER |= GPIO_MODER_MODER7_0;

}
```

Figure 4. Initialization of GPIOB port

2.3.3 GPIOC Initialization

Finally, the initialization process for GPIOC mirrors that of GPIOA to configure the LED pins, as illustrated in Figure 5.

```c
void myGPIOC_Init() {

    /* Enable clock for GPIOC peripheral */
    RCC->AHBENR |= RCC_AHBENR_GPIOCEN;

    /* Configure PC8 and PC9 as outputs */
    GPIOC->MODER |= (GPIO_MODER_MODER8_0 | GPIO_MODER_MODER9_0);

    /* Ensure push-pull mode selected for PC8 and PC9 */
    GPIOC->OTYPER &= ~(GPIO_OTYPER_OT_8 | GPIO_OTYPER_OT_9);

    /* Ensure high-speed mode for PC8 and PC9 */
    GPIOC->OSPEEDR |= (GPIO_OSPEEDER_OSPEEDR8 | GPIO_OSPEEDER_OSPEEDR9);

    /* Ensure no pull-up/pull-down for PC8 and PC9 */
    GPIOC->PUPDR &= ~(GPIO_PUPDR_PUPDR8 | GPIO_PUPDR_PUPDR9);

}
```

Figure 5. Initialization of GPIOC port

2.4 Initialization & Configuration of External Interrupts

In the EXTI initialization function, Figure 6 illustrates the setup for the function generator interrupt. The configuration involves utilizing the *NVIC_SetPriority* and *NVIC_EnableIRQ* functions to define the EXTI lines responsible for triggering the interrupt and to assign a specific priority to the interrupt.

```
//Function Generator Interrupt Configurations:

/* Map EXTI2 line to PA2 */
// Relevant register: SYSCFG->EXTICR[0]
SYSCFG->EXTICR[0] |= SYSCFG_EXTICR1_EXTI2;

/* EXTI2 line interrupts: set rising-edge trigger */
// Relevant register: EXTI->RTSR
EXTI->RTSR |= EXTI_RTSR_TR2;

/* Unmask interrupts from EXTI2 line */
// Relevant register: EXTI->IMR
EXTI->IMR |= EXTI_IMR_MR2;

/* Assign EXTI2 interrupt priority = 0 in NVIC */
// Relevant register: NVIC->IP[2], or use NVIC_SetPriority
NVIC_SetPriority(EXTI2_3_IRQn, 1);

/* Enable EXTI2 interrupts in NVIC */
// Relevant register: NVIC->ISER[0], or use NVIC_EnableIRQ
NVIC_EnableIRQ(EXTI2_3_IRQn);

//End of Function Generator Interrupt Configurations
```

Figure 6. Function generator interrupt configuration

Following that, the configuration for the 555 timer and the user button is executed simultaneously. This configuration is facilitated by their shared use of *EXTI0_1_IRQ*. All interrupts are set to trigger on a rising edge. The identical functions used to establish priority and enable IRQ for the function generator are also utilized for the 555 timer and user button. The comprehensive details of these implementations are illustrated in Figure 7.

```
//555 Timer / User Button Interrupt Configurations:

// Map EXTI1 line to PA1
SYSCFG->EXTICR[0] |= SYSCFG_EXTICR1_EXTI1;

// Map EXTI0 line to PA0 (User Button)
SYSCFG->EXTICR[0] |= SYSCFG_EXTICR1_EXTI0;

// Configure EXTI1 line to interrupt using rising-edge trigger
EXTI->RTSR |= EXTI_RTSR_TR1;

// Configure EXTI0 line to interrupt using rising-edge trigger (User Button)
EXTI->RTSR |= EXTI_RTSR_TR0;

// Enable interrupts from EXTI1 line
EXTI->IMR |= EXTI_IMR_MR1;

// Enable interrupts from EXTI0 line (User Button)
EXTI->IMR |= EXTI_IMR_MR0;

// Assign EXTI1 interrupt priority = 0 (highest) in NVIC
NVIC_SetPriority(EXTI0_1_IRQn, 0);

// Enable EXTI1 interrupts in NVIC
NVIC_EnableIRQ(EXTI0_1_IRQn);
```

Figure 7. Configuration of interrupts for the 555 Timer and User Button

2.5 Analog-to-Digital Converter (ADC) Initialization

The ADC plays a pivotal role in this project, as highlighted in Figure 8. A critical step involves enabling both the SPI1 clock and the ADC clock. Subsequently, the ADC1 configuration registers are utilized to pass the necessary parameters to the ADC. The final steps involve enabling the ADC and implementing a program halt until the ADC is ready, a process achieved through the while loop positioned at the end of the function in Figure 8.

```c
void myADC_Init() {

    // SPI1 clock enable for the ADC
    RCC->APB2ENR |= RCC_APB2ENR_ADCEN;

    // ADC clock enable
    RCC->APB2ENR |= RCC_APB2ENR_ADCEN;

    // Configure ADC for overrun mode
    ADC1->CFGR1 |= ADC_CFGR1_OVRMOD;

    // Configure ADC for continuous mode
    ADC1->CFGR1 |= ADC_CFGR1_CONT;

    // Configure ADC for right-side data alignment
    ADC1->CFGR1 &= ~(ADC_CFGR1_ALIGN);

    // Set ADC to use 12-bit resolution
    ADC1->CFGR1 &= ~(ADC_CFGR1_RES);

    // Configure ADC to select channel 5 for regular sequencer
    ADC1->CHSELR |= ADC_CHSELR_CHSEL5;

    // Configure ADC to use fastest sampling speed
    ADC1->SMPR &= ~(ADC_SMPR_SMP);

    // ADC enable
    ADC1->CR |= ADC_CR_ADEN;
    while ((ADC1->ISR & ADC_ISR_ADRDY) != 1);

}
```

Figure 8. Function for initializing the Analog-to-Digital Converter (ADC)

2.6 Digital-to-Analog Converter (DAC) Initialization

The DAC is initialized with a straightforward process, involving the activation of the clock and Channel 1, as depicted in Figure 9.

```c
void myDAC_Init() {

    // DAC clock enable
    RCC->APB1ENR |= RCC_APB1ENR_DACEN;

    // DAC enable channel 1
    DAC->CR |= DAC_CR_EN1;

}
```

Figure 9. Function for initializing the Digital-to-Analog Converter (DAC)

2.7 Initialization & Configuration of OLED Display

To initiate the OLED screen configuration process, the initial step involves enabling the SPI1 clock to guarantee the screen's optimal performance. Subsequently, the SPI_Handle struct is used to set up crucial parameters such as direction, mode, datasize, and more, as seen in Figure 10. Finally, in the same figure, the procedure demonstrates the explicit action of resetting the OLED screen by setting PB4 to 0. This is executed using the bit set/reset register within GPIOB.

```c
void oled_config( void ) {

// Don't forget to enable GPIOB clock in RCC - COMPLETED (GPIOB_Init Function)
// Don't forget to configure PB3/PB5 as AF0 - COMPLETED (GPIOB_Init Function)
// Don't forget to enable SPI1 clock in RCC - COMPLETED (LINE 500)

    // Enable SPI1 clock in RCC
    RCC->APB2ENR |= RCC_APB2ENR_SPI1EN;

    SPI_Handle.Instance = SPI1;

    SPI_Handle.Init.Direction = SPI_DIRECTION_1LINE;
    SPI_Handle.Init.Mode = SPI_MODE_MASTER;
    SPI_Handle.Init.DataSize = SPI_DATASIZE_8BIT;
    SPI_Handle.Init.CLKPolarity = SPI_POLARITY_LOW;
    SPI_Handle.Init.CLKPhase = SPI_PHASE_1EDGE;
    SPI_Handle.Init.NSS = SPI_NSS_SOFT;
    SPI_Handle.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_256;
    SPI_Handle.Init.FirstBit = SPI_FIRSTBIT_MSB;
    SPI_Handle.Init.CRCPolynomial = 7;

    // Initialize the SPI interface
    HAL_SPI_Init( &SPI_Handle );

    // Enable the SPI
    __HAL_SPI_ENABLE( &SPI_Handle );

    /* Reset LED Display (RES# = PB4):
       - make pin PB4 = 0, wait for a few ms
       - make pin PB4 = 1, wait for a few ms
    */
    GPIOB->BSRR &= ~(GPIO_BSRR_BS_4); // Make pin PB4 = 0
    for(int i=0xF;i>0x0;i--){
        //    sleep(2); // Wait for a few ms - decrementing for loop based on clock speed
    }
```

Figure 10. Detailed overview of the OLED's initial setup, along with reset execution functionality

Next, the OLED display undergoes initialization commands, facilitated by the *Oled_Write_Cmd`* function. Following this initialization, the display is systematically cleared using the nested for loops depicted in Figure 11. The rationale behind using nested for loops is to address each pixel within every row individually, setting them to 0x00. The outer loop guarantees the clearance of every page, while the inner loop ensures the clearance of all 128 pixels within each page.

```
// Send initialization commands to LED Display
for ( unsigned int i = 0; i < sizeof( oled_init_cmds ); i++ )
{
    oled_Write_Cmd( oled_init_cmds[i] );
}

/* Fill LED Display data memory (GDDRAM) with zeros:
   - for each PAGE = 0, 1, ..., 7
       set starting SEG = 0
       call oled_Write_Data( 0x00 ) 128 times
*/
for ( unsigned int i = 0; i < 8; i++) {

    for ( unsigned int j = 0x0; j < 128; j++ ) {

        oled_Write_Cmd((0xB0 | i)); // Page selection 0 - 7
        oled_Write_Cmd(0x00 | (j & 0xF));
        oled_Write_Cmd(0x10 | ((j & 0xF0) >> 4));
        oled_Write_Data(0x00);

    }
}
```

Figure 11. Initialization and clearing loop for the LED Display

Both the *Oled_Write_Cmd* and *Oled_Write_Data* functions share a similar structure in their implementation. In this project, the ODR (Output Data Register) registers were employed to configure the necessary bits for each display element, as illustrated in Figure 12. Although both the ODR and BSRR (Bit Set Reset Register) registers could serve this purpose, the ODR register was chosen for its direct bit-setting capability, allowing for efficient bit manipulation by directly setting the bit to zero.

```
void oled_Write_Cmd( unsigned char cmd ) {

    GPIOB->ODR &= ~(GPIO_ODR_6);// make PB6 = CS# = 0
    GPIOB->ODR &= ~(GPIO_ODR_7); // make PB7 = D/C# = 0
    GPIOB->ODR &= ~(GPIO_ODR_6); // make PB6 = CS# = 0
    oled_Write( cmd );
    GPIOB->ODR |= GPIO_ODR_6; // make PB6 = CS# = 1

}

void oled_Write_Data( unsigned char data ) {

    GPIOB->ODR &= ~(GPIO_ODR_6); // make PB6 = CS# = 0
    GPIOB->ODR |= GPIO_ODR_7; // make PB7 = D/C# = 1
    GPIOB->ODR &= ~(GPIO_ODR_6); // make PB6 = CS# = 0
    oled_Write( data );
    GPIOB->ODR |= GPIO_ODR_6; // make PB6 = CS# = 1

}
```

Figure 12. Oled_Write_Cmd and Oled_Write_Data efficiently transmit commands and data to the OLED display

The *Oled_Write_Data* and *Oled_Write_Cmd* functions both rely on the *Oled_Write* function, as depicted in Figure 13. This function, included in the boilerplate code, employs the *HAL_SPI_Transmit* function after confirming the set state of the SPI1 SR register and the TXE flag. Additionally, before completing the function, it verifies that the SPI1 SR register flag has appropriately reset itself.

```c
void oled_Write( unsigned char Value ) {

    /* Wait until SPI1 is ready for writing (TXE = 1 in SPI1_SR) */
    while (!(SPI1->SR & SPI_SR_TXE));

    /* Send one 8-bit character:
       - This function also sets BIDIOE = 1 in SPI1_CR1
    */
    HAL_SPI_Transmit( &SPI_Handle, &Value, 1, HAL_MAX_DELAY );

    /* Wait until transmission is complete (TXE = 1 in SPI1_SR) */
    while (!(SPI1->SR & SPI_SR_TXE));

}
```

Figure 13. Oled_Write efficiently transmits an 8-bit character using HAL_SPI_Transmit, ensuring readiness and completion with TXE flags in SPI1_SR

To display resistance and frequency values on the OLED screen, the *refresh_OLED* function (Figure 14) is employed. Beginning with the input of resistance characters into the buffer, the function proceeds by using *Oled_Write_Cmd* to set the page and starting column for printing these characters. Subsequently, a nested loop retrieves each character from the buffer, and the ASCII value is utilized to index the character array. The resulting 8 bits of data for each character are then sent individually to the *Oled_Write_Data* function. As the OLED auto-increments with each character written, additional information does not need to be sent to the *Oled_Write_Cmd* function. This entire process is repeated to send frequency data to the OLED.

```
void refresh_OLED( void ) {

    // Buffer size = at most 16 characters per PAGE + terminating '\0'
    unsigned char Buffer[17];

   snprintf( Buffer, sizeof( Buffer ), "R: %5u Ohms", Res );
    /* Buffer now contains your character ASCII codes for LED Display
       - select PAGE (LED Display line) and set starting SEG (column)
       - for each c = ASCII code = Buffer[0], Buffer[1], ...,
           send 8 bytes in Characters[c][0-7] to LED Display
    */
   oled_Write_Cmd(0xB2);
   oled_Write_Cmd(0x03);
   oled_Write_Cmd(0x10);
   for(int i=0;Buffer[i] != '\0';i++){
       for(int j =0;j<8;j++){
           oled_Write_Data(Characters[Buffer[i]][j]);
       }

   }

    snprintf( Buffer, sizeof( Buffer ), "F: %5u Hz", (uint32_t)Freq );
    /* Buffer now contains your character ASCII codes for LED Display
       - select PAGE (LED Display line) and set starting SEG (column)
       - for each c = ASCII code = Buffer[0], Buffer[1], ...,
           send 8 bytes in Characters[c][0-7] to LED Display
    */
    oled_Write_Cmd(0xB5);
    oled_Write_Cmd(0x03);
    oled_Write_Cmd(0x10);
    for(int i=0;Buffer[i] != '\0';i++){
       for(int j =0;j<8;j++){
           oled_Write_Data(Characters[Buffer[i]][j]);
       }

   }

    for(int i=0xFF;i>0x0;i--){ // Delay Timer
    }

}
```

Figure 14. refresh_OLED updates OLED display with resistance and frequency values by implementing the Oled_Write_Cmd and Oled_Write_Data

2.8 Interrupt Handling for EXTI0_1

The *EXTI0_1_IRQHandler* manages interrupts for both the USER button and the 555 timer. In Figure 16, an if statement discerns the flag responsible for the interrupt. If the 555 timer triggers the interrupt, the *calcFreq* function is called. In the case of the USER button, the *inSig* variable undergoes a state change (0x0 to 0x1 or vice versa), depending on its initial state. Subsequently, either the Line 1 interrupt is masked (when transitioning from the 555 timer to the function generator) or the Line 2 interrupt is masked (in the reverse scenario). This masking is achieved through the EXTI Interrupt Mask Register.

Notably, this handler deviates from the typical |= operator for clearing interrupt flags. Instead, the = operator is employed. This unconventional approach is necessary to prevent simultaneous clearing of both the 555 timer and user button flags, which would occur with the |= operator. The = operation in this context serves a unique purpose, compensating for this behavior.

```
void EXTI0_1_IRQHandler() {

    if ((EXTI->PR & EXTI_PR_PR1) != 0 ) {

        calcFreq();

        // Reset interrupt flag
        EXTI->PR = EXTI_PR_PR1;

    }

    if ((EXTI->PR & EXTI_PR_PR0) != 0 ) {
    /* If inSig = 0, then: let inSig = 1, disable
    EXTI1 interrupts, enable EXTI2 interrupts */
    /* Else: let inSig = 0, disable EXTI2 interrupts,
    enable EXTI1 interrupts */

        inSig = (inSig ^ 0x1);

        if ((inSig & 0x1) == 1) {
            // Mask Line 2, unmask Line 1
            EXTI->IMR &= ~(EXTI_IMR_MR2);
            EXTI->IMR |= EXTI_IMR_MR1;
        } else {
            // Mask Line 1, unmask Line 2
            EXTI->IMR &= ~(EXTI_IMR_MR1);
            EXTI->IMR |= EXTI_IMR_MR2;
        }

        // reset interrupt flag
        EXTI->PR = EXTI_PR_PR0;

    }
}
```

Figure 15. EXTI0_1_IRQHandler function manages interrupts for the USER button and the 555 timer

2.9 Interrupt Handling for EXTI2_3

The EXTI2_3 interrupt handler exclusively manages interrupts from the function generator. Upon invocation, it calls the *caclFreq* function, as outlined above in Figure 17. Subsequently, the interrupt flag is promptly reset.

```
void EXTI2_3_IRQHandler() {

    /* Check if EXTI2 interrupt pending flag is indeed set */
    if ((EXTI->PR & EXTI_PR_PR2) != 0) {

        calcFreq();

    }

    EXTI->PR = EXTI_PR_PR2;

}
```

Figure 16. EXTI2_3_IRQHandler function manages interrupts for the function generator

2.10 Interrupt Measurements for 555 Timer & Function Generator

The *calcFreq* function (Figure 15) was implemented to measure the frequency of both the 555 timer and function generator. This function initially discerns whether the interrupt was triggered by the first or second edge by evaluating whether TIM2 was enabled during the interrupt. If it is the first edge, the timer value is reset, and TIM2 is initiated. In the case of the second edge, the TIM2 count value is stored in the variable *measuredCount*. If the *measuredCount* is less than the system clock, the system clock is divided by the *measuredCount* to convert from clock cycles to frequency. Conversely, if the system clock is less than the *measuredCount*, the operation is reversed. This approach ensures accurate frequency calculation even at very low frequencies.

```
void calcFreq() {

    // Initialize local variables for calculations
    float measuredCount = 0.0;

    // 1. This is the second edge:
    if ((TIM2->CR1 & TIM_CR1_CEN) == 1) {

        TIM2->CR1 &= ~(TIM_CR1_CEN); // Stop timer
        measuredCount = TIM2->CNT; // Get timer value

        if (measuredCount < SystemCoreClock) {

            Freq = ((float)SystemCoreClock/measuredCount);// Frequency Calculation

        } else {

            Freq = (measuredCount/(float)SystemCoreClock);// Frequency Calculation

        }

    }

    // 2. This is the first edge:
    else {

        TIM2->CNT = (uint32_t)0x0;

        /* ENABLE TIM2 COUNTER i.e. start counter because input signal detected */
        TIM2->CR1 |= TIM_CR1_CEN;

    }

}
```

Figure 17. The calcFreq function is used to calculate frequency for both the 555 Timer and function generator interrupts

2.11 Main Loop Execution

In the main loop of the program, all initialization functions, as detailed above, are executed. Subsequently, an infinite while loop begins, as depicted in Figure 18. Within the primary while loop, the ADC conversion is initiated using the ADC control register. A brief while loop ensues to confirm the initiation. The ADC conversion flag is then reset, as illustrated in Figure 18. Following this, the variable *pot_measurement* is populated using the ADC DR register. The resistance is calculated by dividing *pot_measurement* by the maximum timeout (0xFFF / 4056) and multiplying it by the maximum value of the potentiometer. The DAC is then provided with the voltage value in the ADC DR register, and the DAC switch trigger is enabled. Finally, the *refresh_OLED* function is called, and the loop repeats.

```c
int
main(int argc, char* argv[]) {

    SystemClock48MHz();

    trace_printf("This is Part 2 of Introductory Lab...\n");
    trace_printf("System clock: %u Hz\n", SystemCoreClock);

    myGPIOA_Init();     /* Initialize I/O port PA */
    myGPIOB_Init();     /* Initialize I/O port PB */
    myADC_Init();       /* Initialize ADC */
    myDAC_Init();       /* Initialize DAC */
    myTIM2_Init();      /* Initialize timer TIM2 */
    myEXTI_Init();      /* Initialize EXTI */
    oled_config();      /* Initialize LED Display */

    while (1) {

        // Start the ADC conversion
        ADC1->CR |= ADC_CR_ADSTART;

        // Ensure the ADC conversion is completed
        while((ADC1->ISR & ADC_ISR_EOC) == 0);

        // Reset ADC conversion flag
        ADC1->ISR &= ~(ADC_ISR_EOC);

        // Get converted ADC value from potentiometer measurement
        pot_measurement = (uint16_t)ADC1->DR & 0x0FFFF;

        // Convert the ADC potentiometer measurement to resistance
        Res = 5000 * ( (pot_measurement) / ((float)0XFFF) );
        // 0xFFF -> 4095 which is the max timeout for the board timers

        trace_printf("ADC Value: %f\n", pot_measurement);



        // Determine voltage value
        input_voltage = (uint16_t)ADC1->DR;

        DAC->DHR12R1 = pot_measurement;

        // Enable switch trigger
        DAC->SWTRIGR |= DAC_SWTRIGR_SWTRIG1;

        // trace_printf(" ", )

        refresh_OLED();

    }

    return 0;
```

Figure 18. The main function initializes system components and enters an infinite loop, continuously conducting various measurements

3. Testing & Results

Each component developed for the final lab presentation underwent individual testing to ensure proper functionality and determine both upper and lower operational limits.

3.1 Frequency Measurement Validation

To assess the 555 timer output, an oscilloscope was employed to visualize the waveform, ensuring that the printed frequency aligned with the observed waveform. The 555 timer frequency exhibited upper and lower limits of approximately 800 Hz to 1.3 kHz as seen in figures 19 and 20. These limits were influenced by the 5 kΩ resistance capacity of the potentiometer and voltage thresholds of 0.7 to 3.3 V for the ADC.



Figure 19: Oscilloscope output signal of the 555 Timer maximum frequency

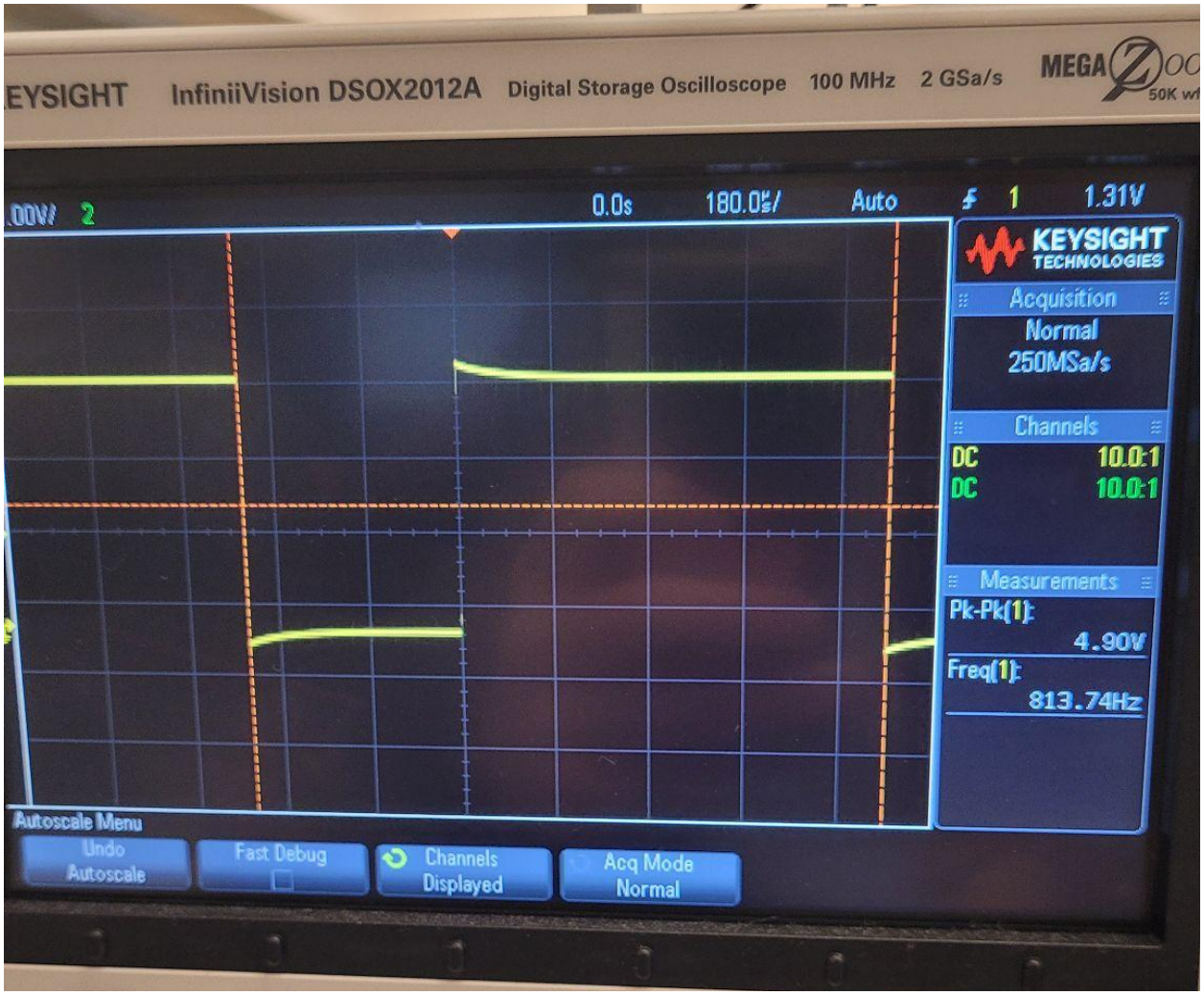


Figure 20. Oscilloscope output signal of the 555 Timer minimum frequency

The frequency measurement for the function generator, established in Lab 2, was previously tested. The upper limit was theoretically 1 MHz, factoring in the error rate within the core clock of the system. Experimental results, however, indicated a maximum frequency around 380 kHz as seen in figures 21 and 22. While the theoretical lower limit, associated with the timer overflow of 0xFFFF, was approximately ~300 Hz as seen in figures 23 and 24.



Figure 21. Experimentally determined maximum frequency (380.3 kHz) for the Function Generator



Figure 22. OLED output display for the experimentally determined maximum frequency for the Function Generator

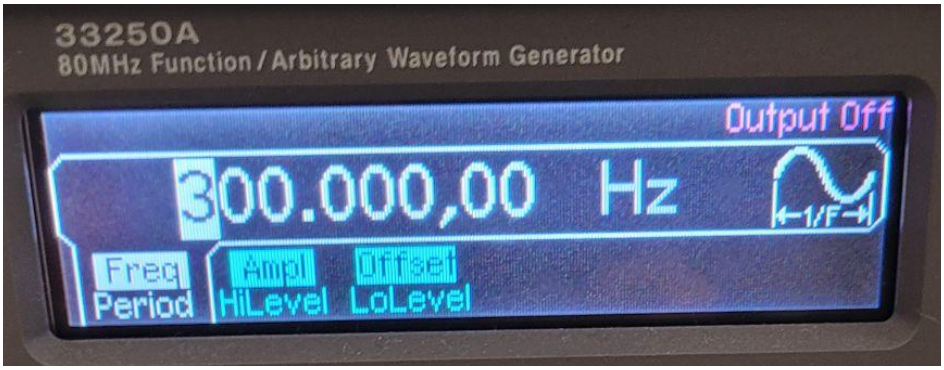


Figure 23. Experimentally determined minimum frequency (300 Hz) for the Function Generator

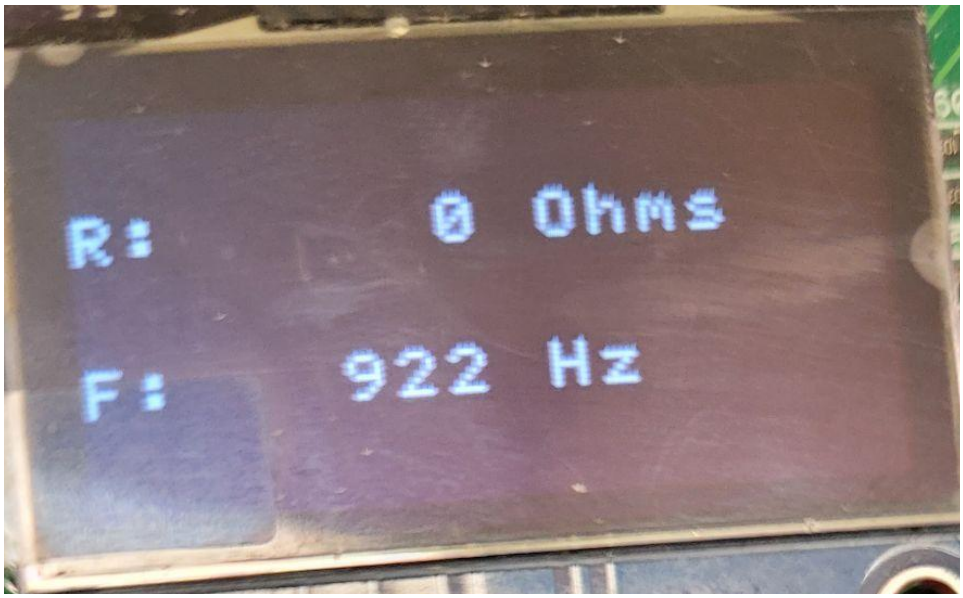


Figure 24. OLED output display for the experimentally determined minimum frequency for the Function Generator

In our system, both waveform frequencies were accurately measured. However, the function generator frequency was slightly lower than the actual signal, registering approximately 14.9 kHz when the generator was set to 15 kHz. Despite this minor discrepancy, it falls within the expected error range.

3.2 Resistance Measurement Validation

The resistance measurement in our system exhibited high accuracy. When assessed using probes from the Digital Multimeter (DMM), the resistance value displayed on the DMM closely matched the value shown on the console. This alignment led us to conclude that the code for converting voltage to resistance was functioning accurately.

3.3 ADC Voltage Validation & Potentiometer Integration

The ADC voltage value was also measured using the oscilloscope, and the peak to peak voltage of the waveform produced was in line with the readings printed in the code using the statement seen in figure 18, these voltages ranged from 0.7 to 3.3 V. The function of the ADC was also combined with the correct resistance readings from the potentiometer which was outlined above.

3.4 DAC Validation & Voltage Comparison

DAC validation involved comparing voltages from PA4 and PA5, where the DAC and ADC were connected, respectively. Successful completion of the test was confirmed when the measured voltages matched. This alignment was achieved by using the ADC data register indirectly, as illustrated in Figure 18, to set the DAC voltage via the *pot_measurement* variable. The minimum and maximum voltages produced by the DAC are shown in figure 25 and 26 being 920mV and 1.97V.

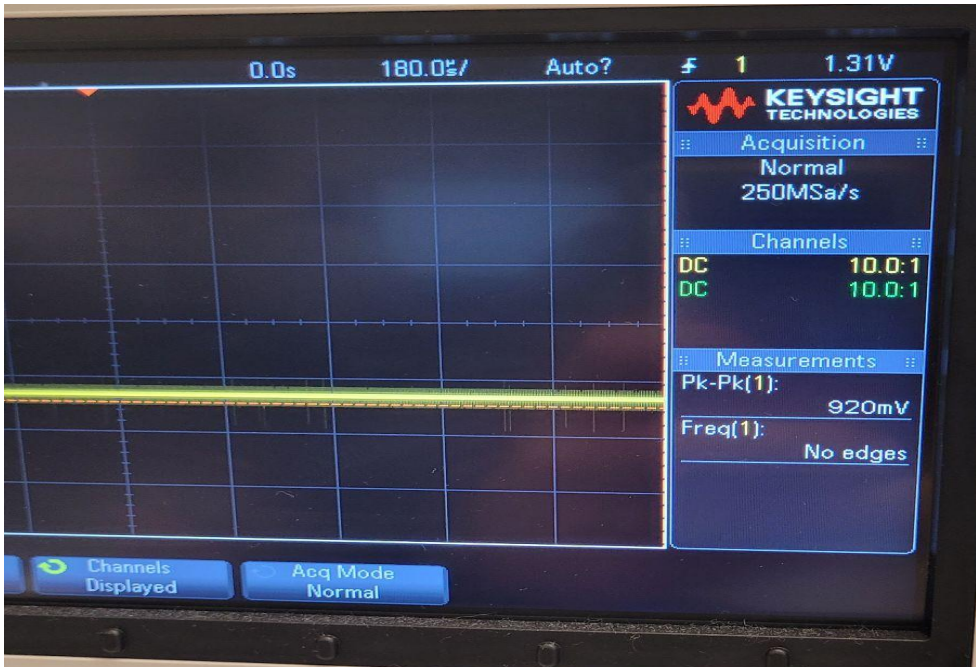


Figure 25. Oscilloscope measurement of minimum DAC voltage output



Figure 26. Oscilloscope measurement of maximum DAC voltage output

3.5 User Button Testing & Corrective Measures

The user button was tested by pressing it to verify the proper switching of frequencies. Initially, an unexpected behavior was observed when measuring the 555 timer frequency, as the |= operator was used to reset the interrupt flag. Upon switching to the = operator, the button functioned correctly, successfully toggling the measured and displayed frequency in the console with each press, confirming its proper operation.

3.6 OLED Display Testing for Frequency and Resistance Measurements

The OLED screen underwent testing after implementing frequency and resistance measurement functionalities. The tests aimed to ensure the accuracy of both the measured frequency and resistance values while continuously updating the display. The minimum and maximum frequency of the 555 timer and resistance of the potentiometer can be shown on the OLED display in figures 27 and 28. Additionally, a startup test verified that the screen was properly cleared in the *Oled_Config* function using the loop shown in Figure 11. Both tests were

successfully passed, as the screen cleared on startup, and the displayed values aligned with the frequency and resistance values printed in the program console.

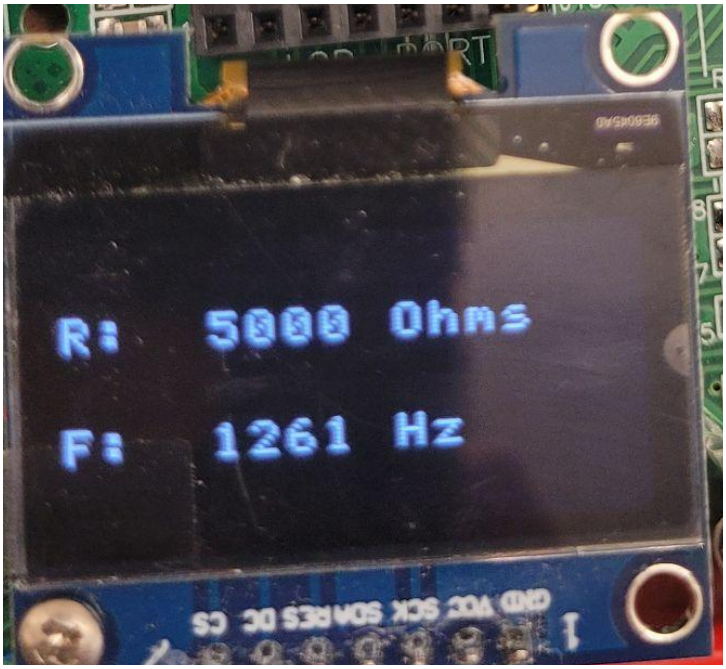


Figure 27: OLED display indicating maximum calculated 555 timer frequency and resistance



Figure 28: OLED Display depicting Minimum Calculated 555 Timer Frequency and Resistance

3.7 System Testing

Upon completing all individual components, we conducted comprehensive testing of the entire system. This involved the following steps:

1. Initiating the system and confirming measurement of the function generator signal.
2. Validating the accuracy of the displayed resistance value.
3. Adjusting the resistance value and confirming that the display accurately reflected the change.
4. Modifying the function generator frequency and confirming alignment with the displayed frequency.
5. Pressing the user button and confirming that the frequency measurement switched to the 555 timer.
6. Adjusting the resistance value and validating that the displayed resistance matched while the frequency display changed accordingly.
7. Pressing the user button multiple times to ensure correct state toggling.

Upon successfully passing these tests, we presented our project to the TA.

4. Discussion

4.1 System Functionality and Objectives

The implementation of the embedded system for monitoring and controlling a PWM signal based on a 555 timer achieved the specified project objectives. The integration of various components such as the ADC, DAC, external interrupts, and SPI communication allowed for a comprehensive and responsive system. The interrupt-driven design, particularly in handling the user button and external signals, showcased the efficient toggling between frequency measurements from the 555 timer and the function generator. The utilization of global variables facilitated the seamless transfer of data between different parts of the system, contributing to its overall functionality.

The integration of SPI communication to configure and drive the LED display (SSD1306 IC) provided a user-friendly interface for visualizing the measured signal frequency and potentiometer resistance. This enhanced the system's accessibility and usability, aligning with the project's objectives to design an efficient and responsive system.

4.2 Hardware Integration and Testing

The testing phase confirmed the proper integration and functionality of individual hardware components. The oscilloscope validation for frequency measurements ensured accuracy within acceptable limits. The alignment of resistance values measured through the ADC with those obtained from a Digital Multimeter (DMM) indicated the precision of the voltage-to-resistance conversion algorithm. Furthermore, the DAC validation and voltage comparison verified the correct operation of the digital-to-analog conversion process.

The corrective measures taken for the user button underscored the importance of meticulous consideration in interrupt handling. The transition from using the |= operator to the = operator rectified unexpected behavior, demonstrating the significance of appropriate programming practices in achieving desired functionality.

4.3 System Robustness and Limitations

The system demonstrated robustness in handling various inputs and adapting to changes in both frequency and resistance. However, it is essential to note the limitations observed during testing. The discrepancy between the theoretical and experimental upper limits of the function generator frequency suggests potential variations in the system's core clock or other external factors affecting the accuracy of frequency measurements.

4.4 Recommendations for Future Enhancements

For future enhancements, refining the frequency measurement accuracy, particularly for the function generator, could be a focus. Calibrating the system to mitigate discrepancies between theoretical and experimental limits would enhance its precision. Additionally, incorporating error-handling mechanisms and feedback mechanisms could further improve the system's reliability and user experience.

5. Conclusion

In conclusion, the development and implementation of the embedded system for monitoring and controlling a PWM signal were successful in meeting the project objectives. The integration of various hardware components, along with a well-designed software architecture, resulted in a responsive and efficient system. The thorough testing process validated the accuracy and reliability of frequency and resistance measurements, while the corrective measures taken during testing underscored the importance of careful consideration in system design.

This project not only provided valuable hands-on experience in microprocessor-based systems but also highlighted the significance of effective collaboration between hardware and software components. The successful demonstration of the system's functionality in the final presentation signifies the achievement of the specified goals and lays the groundwork for potential future enhancements and applications in the realm of embedded systems.

References

[1] D. Rakhmatov, Class Lecture, Topic: "Interface Examples" Faculty of Engineering, Victoria BC: University of Victoria, Fall 2021.

[2] STM32F0xx Reference Manual, 5th revision. STMicroelectronics, Geneva, Switzerland, 2014. Accessed on: December 5th, 2021. [Online]. Available: https://ece.engr.uvic.ca/~ece355/lab/supplement/stm32f0RefManual.pdf

Appendix A: Source Code

```
//
// This file is part of the GNU ARM Eclipse distribution.
// Copyright (c) 2014 Liviu Ionescu.
//

//
-----------------------------------------------------------------------
--
// School: University of Victoria, Canada.
// Course: ECE 355 "Microprocessor-Based Systems".
// This is template code for Part 2 of Introductory Lab.
//
// See "system/include/cmsis/stm32f051x8.h" for register/bit definitions.
// See "system/src/cmsis/vectors_stm32f051x8.c" for handler declarations.
//
-----------------------------------------------------------------------
--

#include <stdio.h>
#include "diag/Trace.h"
#include "cmsis/cmsis_device.h"

//
-----------------------------------------------------------------------
--
//
// STM32F0 empty sample (trace via $(trace)).
//
// Trace support is enabled by adding the TRACE macro definition.
// By default the trace messages are forwarded to the $(trace) output,
// but can be rerouted to any device or completely suppressed, by
// changing the definitions required in system/src/diag/trace_impl.c
// (currently OS_USE_TRACE_ITM, OS_USE_TRACE_SEMIHOSTING_DEBUG/_STDOUT).
//

// ----- main()
-------------------------------------------------------------
```

```
// Sample pragmas to cope with warnings. Please note the related line at
// the end of this function, used to pop the compiler diagnostics status.
#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wunused-parameter"
#pragma GCC diagnostic ignored "-Wmissing-declarations"
#pragma GCC diagnostic ignored "-Wreturn-type"


/* Definitions of registers and their bits are
   given in system/include/cmsis/stm32f051x8.h */


/* Clock prescaler for TIM2 timer: no prescaling */
#define myTIM2_PRESCALER ((uint16_t)0x0000)
/* Maximum possible setting for overflow */
#define myTIM2_PERIOD ((uint32_t)0xFFFFFFFF)

void myGPIOA_Init(void);
void myGPIOB_Init(void);
void myADC_Init(void);
void myDAC_Init(void);
void myTIM2_Init(void);
void myEXTI_Init(void);

// Initialize global variables
float Freq = 6969;  // measured frequency value (global variable)
unsigned int Res = 6969;   // measured resistance value (global variable)

float input_voltage = 0;
float dac_voltage = 0;
float pot_measurement = 0;

int inSig = 0;

// Start of SPI set up

void oled_Write(unsigned char);
void oled_Write_Cmd(unsigned char);
void oled_Write_Data(unsigned char);
```

```
void oled_config(void);

void refresh_OLED(void);

SPI_HandleTypeDef SPI_Handle;

// LED Display initialization commands
unsigned char oled_init_cmds[] =
{
    0xAE,
    0x20, 0x00,
    0x40,
    0xA0 | 0x01,
    0xA8, 0x40 - 1,
    0xC0 | 0x08,
    0xD3, 0x00,
    0xDA, 0x32,
    0xD5, 0x80,
    0xD9, 0x22,
    0xDB, 0x30,
    0x81, 0xFF,
    0xA4,
    0xA6,
    0xAD, 0x30,
    0x8D, 0x10,
    0xAE | 0x01,
    0xC0,
    0xA0
};

// Character specifications for LED Display (1 row = 8 bytes = 1 ASCII
character)
// Example: to display '4', retrieve 8 data bytes stored in
Characters[52][X] row
//          (where X = 0, 1, ..., 7) and send them one by one to LED
Display.
// Row number = character ASCII code (e.g., ASCII code of '4' is 0x34 =
52)
unsigned char Characters[][8] = {
```

```
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
```

```
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b01011111, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // !
    {0b00000000, 0b00000111, 0b00000000, 0b00000111,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // "
    {0b00010100, 0b01111111, 0b00010100, 0b01111111,
0b00010100,0b00000000, 0b00000000, 0b00000000},  // #
    {0b00100100, 0b00101010, 0b01111111, 0b00101010,
0b00010010,0b00000000, 0b00000000, 0b00000000},  // $
    {0b00100011, 0b00010011, 0b00001000, 0b01100100,
0b01100010,0b00000000, 0b00000000, 0b00000000},  // %
    {0b00110110, 0b01001001, 0b01010101, 0b00100010,
0b01010000,0b00000000, 0b00000000, 0b00000000},  // &
    {0b00000000, 0b00000101, 0b00000011, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // '
```

```
    {0b00000000, 0b00011100, 0b00100010, 0b01000001,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // (
    {0b00000000, 0b01000001, 0b00100010, 0b00011100,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // )
    {0b00010100, 0b00001000, 0b00111110, 0b00001000,
0b00010100,0b00000000, 0b00000000, 0b00000000},  // *
    {0b00001000, 0b00001000, 0b00111110, 0b00001000,
0b00001000,0b00000000, 0b00000000, 0b00000000},  // +
    {0b00000000, 0b01010000, 0b00110000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // ,
    {0b00001000, 0b00001000, 0b00001000, 0b00001000,
0b00001000,0b00000000, 0b00000000, 0b00000000},  // -
    {0b00000000, 0b01100000, 0b01100000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // .
    {0b00100000, 0b00010000, 0b00001000, 0b00000100,
0b00000010,0b00000000, 0b00000000, 0b00000000},  // /
    {0b00111110, 0b01010001, 0b01001001, 0b01000101,
0b00111110,0b00000000, 0b00000000, 0b00000000},  // 0
    {0b00000000, 0b01000010, 0b01111111, 0b01000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // 1
    {0b01000010, 0b01100001, 0b01010001, 0b01001001,
0b01000110,0b00000000, 0b00000000, 0b00000000},  // 2
    {0b00100001, 0b01000001, 0b01000101, 0b01001011,
0b00110001,0b00000000, 0b00000000, 0b00000000},  // 3
    {0b00011000, 0b00010100, 0b00010010, 0b01111111,
0b00010000,0b00000000, 0b00000000, 0b00000000},  // 4
    {0b00100111, 0b01000101, 0b01000101, 0b01000101,
0b00111001,0b00000000, 0b00000000, 0b00000000},  // 5
    {0b00111100, 0b01001010, 0b01001001, 0b01001001,
0b00110000,0b00000000, 0b00000000, 0b00000000},  // 6
    {0b00000011, 0b00000001, 0b01110001, 0b00001001,
0b00000111,0b00000000, 0b00000000, 0b00000000},  // 7
    {0b00110110, 0b01001001, 0b01001001, 0b01001001,
0b00110110,0b00000000, 0b00000000, 0b00000000},  // 8
    {0b00000110, 0b01001001, 0b01001001, 0b00101001,
0b00011110,0b00000000, 0b00000000, 0b00000000},  // 9
    {0b00000000, 0b00110110, 0b00110110, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // :
    {0b00000000, 0b01010110, 0b00110110, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // ;
```

```
    {0b00001000, 0b00010100, 0b00100010, 0b01000001,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // <
    {0b00010100, 0b00010100, 0b00010100, 0b00010100,
0b00010100,0b00000000, 0b00000000, 0b00000000},  // =
    {0b00000000, 0b01000001, 0b00100010, 0b00010100,
0b00001000,0b00000000, 0b00000000, 0b00000000},  // >
    {0b00000010, 0b00000001, 0b01010001, 0b00001001,
0b00000110,0b00000000, 0b00000000, 0b00000000},  // ?
    {0b00110010, 0b01001001, 0b01111001, 0b01000001,
0b00111110,0b00000000, 0b00000000, 0b00000000},  // @
    {0b01111110, 0b00010001, 0b00010001, 0b00010001,
0b01111110,0b00000000, 0b00000000, 0b00000000},  // A
    {0b01111111, 0b01001001, 0b01001001, 0b01001001,
0b00110110,0b00000000, 0b00000000, 0b00000000},  // B
    {0b00111110, 0b01000001, 0b01000001, 0b01000001,
0b00100010,0b00000000, 0b00000000, 0b00000000},  // C
    {0b01111111, 0b01000001, 0b01000001, 0b00100010,
0b00011100,0b00000000, 0b00000000, 0b00000000},  // D
    {0b01111111, 0b01001001, 0b01001001, 0b01001001,
0b01000001,0b00000000, 0b00000000, 0b00000000},  // E
    {0b01111111, 0b00001001, 0b00001001, 0b00001001,
0b00000001,0b00000000, 0b00000000, 0b00000000},  // F
    {0b00111110, 0b01000001, 0b01001001, 0b01001001,
0b01111010,0b00000000, 0b00000000, 0b00000000},  // G
    {0b01111111, 0b00001000, 0b00001000, 0b00001000,
0b01111111,0b00000000, 0b00000000, 0b00000000},  // H
    {0b01000000, 0b01000001, 0b01111111, 0b01000001,
0b01000000,0b00000000, 0b00000000, 0b00000000},  // I
    {0b00100000, 0b01000000, 0b01000001, 0b00111111,
0b00000001,0b00000000, 0b00000000, 0b00000000},  // J
    {0b01111111, 0b00001000, 0b00010100, 0b00100010,
0b01000001,0b00000000, 0b00000000, 0b00000000},  // K
    {0b01111111, 0b01000000, 0b01000000, 0b01000000,
0b01000000,0b00000000, 0b00000000, 0b00000000},  // L
    {0b01111111, 0b00000010, 0b00001100, 0b00000010,
0b01111111,0b00000000, 0b00000000, 0b00000000},  // M
    {0b01111111, 0b00000100, 0b00001000, 0b00010000,
0b01111111,0b00000000, 0b00000000, 0b00000000},  // N
    {0b00111110, 0b01000001, 0b01000001, 0b01000001,
0b00111110,0b00000000, 0b00000000, 0b00000000},  // O
```

```
    {0b01111111, 0b00001001, 0b00001001, 0b00001001,
0b00000110,0b00000000, 0b00000000, 0b00000000},  // P
    {0b00111110, 0b01000001, 0b01010001, 0b00100001,
0b01011110,0b00000000, 0b00000000, 0b00000000},  // Q
    {0b01111111, 0b00001001, 0b00011001, 0b00101001,
0b01000110,0b00000000, 0b00000000, 0b00000000},  // R
    {0b01000110, 0b01001001, 0b01001001, 0b01001001,
0b00110001,0b00000000, 0b00000000, 0b00000000},  // S
    {0b00000001, 0b00000001, 0b01111111, 0b00000001,
0b00000001,0b00000000, 0b00000000, 0b00000000},  // T
    {0b00111111, 0b01000000, 0b01000000, 0b01000000,
0b00111111,0b00000000, 0b00000000, 0b00000000},  // U
    {0b00011111, 0b00100000, 0b01000000, 0b00100000,
0b00011111,0b00000000, 0b00000000, 0b00000000},  // V
    {0b00111111, 0b01000000, 0b00111000, 0b01000000,
0b00111111,0b00000000, 0b00000000, 0b00000000},  // W
    {0b01100011, 0b00010100, 0b00001000, 0b00010100,
0b01100011,0b00000000, 0b00000000, 0b00000000},  // X
    {0b00000111, 0b00001000, 0b01110000, 0b00001000,
0b00000111,0b00000000, 0b00000000, 0b00000000},  // Y
    {0b01100001, 0b01010001, 0b01001001, 0b01000101,
0b01000011,0b00000000, 0b00000000, 0b00000000},  // Z
    {0b01111111, 0b01000001, 0b00000000, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // [
    {0b00010101, 0b00010110, 0b01111100, 0b00010110,
0b00010101,0b00000000, 0b00000000, 0b00000000},  // back slash
    {0b00000000, 0b00000000, 0b00000000, 0b01000001,
0b01111111,0b00000000, 0b00000000, 0b00000000},  // ]
    {0b00000100, 0b00000010, 0b00000001, 0b00000010,
0b00000100,0b00000000, 0b00000000, 0b00000000},  // ^
    {0b01000000, 0b01000000, 0b01000000, 0b01000000,
0b01000000,0b00000000, 0b00000000, 0b00000000},  // _
    {0b00000000, 0b00000001, 0b00000010, 0b00000100,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // `
    {0b00100000, 0b01010100, 0b01010100, 0b01010100,
0b01111000,0b00000000, 0b00000000, 0b00000000},  // a
    {0b01111111, 0b01001000, 0b01000100, 0b01000100,
0b00111000,0b00000000, 0b00000000, 0b00000000},  // b
    {0b00111000, 0b01000100, 0b01000100, 0b01000100,
0b00100000,0b00000000, 0b00000000, 0b00000000},  // c
```

```
    {0b00111000, 0b01000100, 0b01000100, 0b01001000,
0b01111111,0b00000000, 0b00000000, 0b00000000},  // d
    {0b00111000, 0b01010100, 0b01010100, 0b01010100,
0b00011000,0b00000000, 0b00000000, 0b00000000},  // e
    {0b00001000, 0b01111110, 0b00001001, 0b00000001,
0b00000010,0b00000000, 0b00000000, 0b00000000},  // f
    {0b00001100, 0b01010010, 0b01010010, 0b01010010,
0b00111110,0b00000000, 0b00000000, 0b00000000},  // g
    {0b01111111, 0b00001000, 0b00000100, 0b00000100,
0b01111000,0b00000000, 0b00000000, 0b00000000},  // h
    {0b00000000, 0b01000100, 0b01111101, 0b01000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // i
    {0b00100000, 0b01000000, 0b01000100, 0b00111101,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // j
    {0b01111111, 0b00010000, 0b00101000, 0b01000100,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // k
    {0b00000000, 0b01000001, 0b01111111, 0b01000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // l
    {0b01111100, 0b00000100, 0b00011000, 0b00000100,
0b01111000,0b00000000, 0b00000000, 0b00000000},  // m
    {0b01111100, 0b00001000, 0b00000100, 0b00000100,
0b01111000,0b00000000, 0b00000000, 0b00000000},  // n
    {0b00111000, 0b01000100, 0b01000100, 0b01000100,
0b00111000,0b00000000, 0b00000000, 0b00000000},  // o
    {0b01111100, 0b00010100, 0b00010100, 0b00010100,
0b00001000,0b00000000, 0b00000000, 0b00000000},  // p
    {0b00001000, 0b00010100, 0b00010100, 0b00011000,
0b01111100,0b00000000, 0b00000000, 0b00000000},  // q
    {0b01111100, 0b00001000, 0b00000100, 0b00000100,
0b00001000,0b00000000, 0b00000000, 0b00000000},  // r
    {0b01001000, 0b01010100, 0b01010100, 0b01010100,
0b00100000,0b00000000, 0b00000000, 0b00000000},  // s
    {0b00000100, 0b00111111, 0b01000100, 0b01000000,
0b00100000,0b00000000, 0b00000000, 0b00000000},  // t
    {0b00111100, 0b01000000, 0b01000000, 0b00100000,
0b01111100,0b00000000, 0b00000000, 0b00000000},  // u
    {0b00011100, 0b00100000, 0b01000000, 0b00100000,
0b00011100,0b00000000, 0b00000000, 0b00000000},  // v
    {0b00111100, 0b01000000, 0b00111000, 0b01000000,
0b00111100,0b00000000, 0b00000000, 0b00000000},  // w
```

```
    {0b01000100, 0b00101000, 0b00010000, 0b00101000,
0b01000100,0b00000000, 0b00000000, 0b00000000},  // x
    {0b00001100, 0b01010000, 0b01010000, 0b01010000,
0b00111100,0b00000000, 0b00000000, 0b00000000},  // y
    {0b01000100, 0b01100100, 0b01010100, 0b01001100,
0b01000100,0b00000000, 0b00000000, 0b00000000},  // z
    {0b00000000, 0b00001000, 0b00110110, 0b01000001,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // {
    {0b00000000, 0b00000000, 0b01111111, 0b00000000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // |
    {0b00000000, 0b01000001, 0b00110110, 0b00001000,
0b00000000,0b00000000, 0b00000000, 0b00000000},  // }
    {0b00001000, 0b00001000, 0b00101010, 0b00011100,
0b00001000,0b00000000, 0b00000000, 0b00000000},  // ~
    {0b00001000, 0b00011100, 0b00101010, 0b00001000,
0b00001000,0b00000000, 0b00000000, 0b00000000}   // <-
};

// END of spi set up

/*** Call this function to boost the STM32F0xx clock to 48 MHz ***/
void SystemClock48MHz( void ) {

    // Disable the PLL
    RCC->CR &= ~(RCC_CR_PLLON);

    // Wait for the PLL to unlock
    while (( RCC->CR & RCC_CR_PLLRDY ) != 0 );

    // Configure the PLL for 48-MHz system clock
    RCC->CFGR = 0x00280000;

    // Enable the PLL
    RCC->CR |= RCC_CR_PLLON;

    // Wait for the PLL to lock
    while (( RCC->CR & RCC_CR_PLLRDY ) != RCC_CR_PLLRDY );

    // Switch the processor to the PLL clock source
    RCC->CFGR = ( RCC->CFGR & (~RCC_CFGR_SW_Msk)) | RCC_CFGR_SW_PLL;
```

```
    // Update the system with the new clock Freq
    SystemCoreClockUpdate();

}


/*****************************************************************/


int
main(int argc, char* argv[]) {

    SystemClock48MHz();

    trace_printf("This is Part 2 of Introductory Lab...\n");
    trace_printf("System clock: %u Hz\n", SystemCoreClock);

    myGPIOA_Init();     /* Initialize I/O port PA */
    myGPIOB_Init();     /* Initialize I/O port PB */
    myADC_Init();       /* Initialize ADC */
    myDAC_Init();       /* Initialize DAC */
    myTIM2_Init();      /* Initialize timer TIM2 */
    myEXTI_Init();      /* Initialize EXTI */
    oled_config();      /* Initialize LED Display */

    while (1) {

        // Start the ADC conversion
        ADC1->CR |= ADC_CR_ADSTART;

        // Ensure the ADC conversion is completed
        while((ADC1->ISR & ADC_ISR_EOC) == 0);

        // Reset ADC conversion flag
        ADC1->ISR &= ~(ADC_ISR_EOC);

        // Get converted ADC value from potentiometer measurement
        pot_measurement = (uint16_t)ADC1->DR & 0x0FFF;
```

```
        // Convert the ADC potentiometer measurement to resistance
        Res = 5000 * ( (pot_measurement) / ((float)0XFFF) );
        // 0xFFF -> 4095 which is the max timeout for the board timers


        // Determine voltage value
        input_voltage = (uint16_t)ADC1->DR;

        DAC->DHR12R1 = pot_measurement;

        // Enable switch trigger
        DAC->SWTRIGR |= DAC_SWTRIGR_SWTRIG1;


        refresh_OLED();


    }

    return 0;

}


// LED Display Functions

void refresh_OLED( void ) {

    // Buffer size = at most 16 characters per PAGE + terminating '\0'
    unsigned char Buffer[17];

   snprintf( Buffer, sizeof( Buffer ), "R: %5u Ohms", Res );
    /* Buffer now contains your character ASCII codes for LED Display
       - select PAGE (LED Display line) and set starting SEG (column)
       - for each c = ASCII code = Buffer[0], Buffer[1], ...,
           send 8 bytes in Characters[c][0-7] to LED Display
    */
   oled_Write_Cmd(0xB2);
   oled_Write_Cmd(0x03);
   oled_Write_Cmd(0x10);
```

```
   for(int i=0;Buffer[i] != '\0';i++){
       for(int j =0;j<8;j++){
           oled_Write_Data(Characters[Buffer[i]][j]);
       }
   }

    snprintf( Buffer, sizeof( Buffer ), "F: %5u Hz", (uint32_t)Freq );
    /* Buffer now contains your character ASCII codes for LED Display
       - select PAGE (LED Display line) and set starting SEG (column)
       - for each c = ASCII code = Buffer[0], Buffer[1], ...,
           send 8 bytes in Characters[c][0-7] to LED Display
    */
    oled_Write_Cmd(0xB5);
    oled_Write_Cmd(0x03);
    oled_Write_Cmd(0x10);
    for(int i=0;Buffer[i] != '\0';i++){
       for(int j =0;j<8;j++){
           oled_Write_Data(Characters[Buffer[i]][j]);
       }
   }

    for(int i=0xFF;i>0x0;i--){ // Delay Timer
    }

}

void oled_Write_Cmd( unsigned char cmd ) {

    GPIOB->ODR &= ~(GPIO_ODR_6);// make PB6 = CS# = 0
    GPIOB->ODR &= ~(GPIO_ODR_7); // make PB7 = D/C# = 0
    GPIOB->ODR &= ~(GPIO_ODR_6); // make PB6 = CS# = 0
    oled_Write( cmd );
    GPIOB->ODR |= GPIO_ODR_6; // make PB6 = CS# = 1

}

void oled_Write_Data( unsigned char data ) {

    GPIOB->ODR &= ~(GPIO_ODR_6); // make PB6 = CS# = 0
    GPIOB->ODR |= GPIO_ODR_7; // make PB7 = D/C# = 1
```

```
    GPIOB->ODR &= ~(GPIO_ODR_6); // make PB6 = CS# = 0
    oled_Write( data );
    GPIOB->ODR |= GPIO_ODR_6; // make PB6 = CS# = 1

}

void oled_Write( unsigned char Value ) {

    /* Wait until SPI1 is ready for writing (TXE = 1 in SPI1_SR) */
    while (!(SPI1->SR & SPI_SR_TXE));

    /* Send one 8-bit character:
       - This function also sets BIDIOE = 1 in SPI1_CR1
    */
    HAL_SPI_Transmit( &SPI_Handle, &Value, 1, HAL_MAX_DELAY );

    /* Wait until transmission is complete (TXE = 1 in SPI1_SR) */
    while (!(SPI1->SR & SPI_SR_TXE));

}

void oled_config( void ) {

// Don't forget to enable GPIOB clock in RCC - COMPLETED (GPIOB_Init
Function)
// Don't forget to configure PB3/PB5 as AF0 - COMPLETED (GPIOB_Init
Function)
// Don't forget to enable SPI1 clock in RCC - COMPLETED (LINE 500)

    // Enable SPI1 clock in RCC
    RCC->APB2ENR |= RCC_APB2ENR_SPI1EN;

    SPI_Handle.Instance = SPI1;

    SPI_Handle.Init.Direction = SPI_DIRECTION_1LINE;
    SPI_Handle.Init.Mode = SPI_MODE_MASTER;
    SPI_Handle.Init.DataSize = SPI_DATASIZE_8BIT;
    SPI_Handle.Init.CLKPolarity = SPI_POLARITY_LOW;
    SPI_Handle.Init.CLKPhase = SPI_PHASE_1EDGE;
    SPI_Handle.Init.NSS = SPI_NSS_SOFT;
```

```
    SPI_Handle.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_256;
    SPI_Handle.Init.FirstBit = SPI_FIRSTBIT_MSB;
    SPI_Handle.Init.CRCPolynomial = 7;

    // Initialize the SPI interface
    HAL_SPI_Init( &SPI_Handle );

    // Enable the SPI
    __HAL_SPI_ENABLE( &SPI_Handle );

    /* Reset LED Display (RES# = PB4):
       - make pin PB4 = 0, wait for a few ms
       - make pin PB4 = 1, wait for a few ms
    */
    GPIOB->BSRR &= ~(GPIO_BSRR_BS_4); // Make pin PB4 = 0
    for(int i=0xF;i>0x0;i--){
        //    sleep(2); // Wait for a few ms - decrementing for loop based
on clock speed
    }

    GPIOB->BSRR = GPIO_BSRR_BS_4; // Setting Pin PB4 = 1
    for(int i=0xF;i>0x0;i--){
        //  sleep(2); // Wait for a few ms - decrementing for loop based
on clock speed
    }


    // Send initialization commands to LED Display
    for ( unsigned int i = 0; i < sizeof( oled_init_cmds ); i++ )
    {
        oled_Write_Cmd( oled_init_cmds[i] );
    }

    /* Fill LED Display data memory (GDDRAM) with zeros:
       - for each PAGE = 0, 1, ..., 7
           set starting SEG = 0
           call oled_Write_Data( 0x00 ) 128 times
    */
    for ( unsigned int i = 0; i < 8; i++) {
```

```
        for ( unsigned int j = 0x0; j < 128; j++ ) {

            oled_Write_Cmd((0xB0 | i)); // Page selection 0 - 7
            oled_Write_Cmd(0x00 | (j & 0xF));
            oled_Write_Cmd(0x10 | ((j & 0xF0) >> 4));
            oled_Write_Data(0x00);

        }
    }
}

// End of LED Display Functions

void myGPIOA_Init() {

    /* Enable clock for GPIOA peripheral */
    RCC->AHBENR |= RCC_AHBENR_GPIOAEN;

    /* Configure PA2 as input */
    GPIOA->MODER &= ~(GPIO_MODER_MODER2);

    /* Ensure no pull-up/pull-down for PA2 */
    GPIOA->PUPDR &= ~(GPIO_PUPDR_PUPDR2);

    /* Configure PA4 as input */
    GPIOA->MODER |= GPIO_MODER_MODER4;

    /* Ensure no pull-up/pull-down for PA4 */
    GPIOA->PUPDR &= ~(GPIO_PUPDR_PUPDR4);


    // User Button Configuration:

    /* Configure PA0 as input */
    GPIOA->MODER &= ~(GPIO_MODER_MODER0);

    /* Ensure no pull-up/pull-down for PA0 */
    GPIOA->PUPDR &= ~(GPIO_PUPDR_PUPDR0);

}
```

```
void myGPIOB_Init() {

    /* Enable clock for GPIOB peripheral */
    RCC->AHBENR |= RCC_AHBENR_GPIOBEN;

    /* Configure PB3 to use alternate function SCK */
    GPIOB->MODER |= GPIO_MODER_MODER3_1;

    /* Ensure no pull-up/pull-down for PB3 */
    GPIOB->PUPDR &= ~(GPIO_PUPDR_PUPDR3);

    /* Configure PB4 as RES# (Reset) */
    GPIOB->MODER |= GPIO_MODER_MODER4_0;

    /* Ensure no pull-up/pull-down for PB4 */
    GPIOB->PUPDR &= ~(GPIO_PUPDR_PUPDR4);

    /* Configure PB5 as output MOSI (Serial Data) */
    GPIOB->MODER |= GPIO_MODER_MODER5_1;

    /* Ensure no pull-up/pull-down for PB5 */
    GPIOB->PUPDR &= ~(GPIO_PUPDR_PUPDR5);

    /* Configure board to recognize PB5 is being used as alternate
function SCK (Serial Clock) */
    GPIOB->AFR[0] |= GPIO_AFRL_AFSEL0;

    /* Configure PB6 as CS# (Chip Select) */
    GPIOB->MODER |= GPIO_MODER_MODER6_0;

    /* Configure PB7 as D/C# (Data/Command) */
    GPIOB->MODER |= GPIO_MODER_MODER7_0;

}

void myGPIOC_Init() {

    /* Enable clock for GPIOC peripheral */
    RCC->AHBENR |= RCC_AHBENR_GPIOCEN;
```

```
    /* Configure PC8 and PC9 as outputs */
    GPIOC->MODER |= (GPIO_MODER_MODER8_0 | GPIO_MODER_MODER9_0);

    /* Ensure push-pull mode selected for PC8 and PC9 */
    GPIOC->OTYPER &= ~(GPIO_OTYPER_OT_8 | GPIO_OTYPER_OT_9);

    /* Ensure high-speed mode for PC8 and PC9 */
    GPIOC->OSPEEDR |= (GPIO_OSPEEDER_OSPEEDR8 | GPIO_OSPEEDER_OSPEEDR9);

    /* Ensure no pull-up/pull-down for PC8 and PC9 */
    GPIOC->PUPDR &= ~(GPIO_PUPDR_PUPDR8 | GPIO_PUPDR_PUPDR9);

}

void myADC_Init() {

    // SPI1 clock enable for the ADC
    RCC->APB2ENR |= RCC_APB2ENR_ADCEN;

    // ADC clock enable
    RCC->APB2ENR |= RCC_APB2ENR_ADCEN;

    // Configure ADC for overrun mode
    ADC1->CFGR1 |= ADC_CFGR1_OVRMOD;

    // Configure ADC for continuous mode
    ADC1->CFGR1 |= ADC_CFGR1_CONT;

    // Configure ADC for right-side data alignment
    ADC1->CFGR1 &= ~(ADC_CFGR1_ALIGN);

    // Set ADC to use 12-bit resolution
    ADC1->CFGR1 &= ~(ADC_CFGR1_RES);

    // Configure ADC to select channel 5 for regular sequencer
    ADC1->CHSELR |= ADC_CHSELR_CHSEL5;

    // Configure ADC to use fastest sampling speed
    ADC1->SMPR &= ~(ADC_SMPR_SMP);
```

```
    // ADC enable
    ADC1->CR |= ADC_CR_ADEN;
    while ((ADC1->ISR & ADC_ISR_ADRDY) != 1);

}

void myDAC_Init() {

    // DAC clock enable
    RCC->APB1ENR |= RCC_APB1ENR_DACEN;

    // DAC enable channel 1
    DAC->CR |= DAC_CR_EN1;

}

void myTIM2_Init() {

    // Enable clock for TIM2 peripheral
    RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;

    // Configure TIM2: buffer auto-reload, count up, stop on overflow,
    // enable update events, interrupt on overflow only
    TIM2->ARR = myTIM2_PERIOD;
    TIM2->CR1 |= (uint16_t)0x8C;
    TIM2->CR1 &= (uint16_t)0xFF8C;

    // Set clock prescaler value
    TIM2->PSC = myTIM2_PRESCALER;

    // Update timer registers with above changes
    TIM2->EGR |= TIM_EGR_UG;

    // Assign TIM2 interrupt priority = 0 (highest) in NVIC
    NVIC_SetPriority(15, 0);

    // Enable TIM2 interrupts in NVIC
    NVIC_EnableIRQ(15);
```

```
    // Enable TIM2 interrupt generation
    TIM2->DIER |= TIM_DIER_UIE;

}

void myEXTI_Init() {

    //Function Generator Interrupt Configurations:

    /* Map EXTI2 line to PA2 */
    // Relevant register: SYSCFG->EXTICR[0]
    SYSCFG->EXTICR[0] |= SYSCFG_EXTICR1_EXTI2;

    /* EXTI2 line interrupts: set rising-edge trigger */
    // Relevant register: EXTI->RTSR
    EXTI->RTSR |= EXTI_RTSR_TR2;

    /* Unmask interrupts from EXTI2 line */
    // Relevant register: EXTI->IMR
    EXTI->IMR |= EXTI_IMR_MR2;

    /* Assign EXTI2 interrupt priority = 0 in NVIC */
    // Relevant register: NVIC->IP[2], or use NVIC_SetPriority
    NVIC_SetPriority(EXTI2_3_IRQn, 1);

    /* Enable EXTI2 interrupts in NVIC */
    // Relevant register: NVIC->ISER[0], or use NVIC_EnableIRQ
    NVIC_EnableIRQ(EXTI2_3_IRQn);

    //End of Function Generator Interrupt Configurations



    //555 Timer / User Button Interrupt Configurations:

    // Map EXTI1 line to PA1
    SYSCFG->EXTICR[0] |= SYSCFG_EXTICR1_EXTI1;

    // Map EXTI0 line to PA0 (User Button)
    SYSCFG->EXTICR[0] |= SYSCFG_EXTICR1_EXTI0;
```

```
    // Configure EXTI1 line to interrupt using rising-edge trigger
    EXTI->RTSR |= EXTI_RTSR_TR1;

    // Configure EXTI0 line to interrupt using rising-edge trigger (User
Button)
    EXTI->RTSR |= EXTI_RTSR_TR0;

    // Enable interrupts from EXTI1 line
    EXTI->IMR |= EXTI_IMR_MR1;

    // Enable interrupts from EXTI0 line (User Button)
    EXTI->IMR |= EXTI_IMR_MR0;

    // Assign EXTI1 interrupt priority = 0 (highest) in NVIC
    NVIC_SetPriority(EXTI0_1_IRQn, 0);

    // Enable EXTI1 interrupts in NVIC
    NVIC_EnableIRQ(EXTI0_1_IRQn);

    //End of 555 Timer / User Button Interrupt Configurations

    EXTI->IMR &= ~(EXTI_IMR_MR1);
    EXTI->IMR |= EXTI_IMR_MR2;

}

/* This handler is declared in system/src/cmsis/vectors_stm32f051x8.c */
void TIM2_IRQHandler() {

    // Check that TIM2 interrupt flag is set
    if ((TIM2->SR & TIM_SR_UIF) != 0) {

        // TIM2 overflow has occurred
        trace_printf("\n*** Overflow! ***\n");

        // Clear interrupt flag
        TIM2->SR &= (uint32_t)0xFFFE;

        // Restart stopped timer
```

```
        TIM2->CR1 |= (uint16_t)0x1;

    }

}

void calcFreq() {

    // Initialize local variables for calculations
    float measuredCount = 0.0;

    // 1. This is the second edge:
    if ((TIM2->CR1 & TIM_CR1_CEN) == 1) {

        TIM2->CR1 &= ~(TIM_CR1_CEN); // Stop timer
        measuredCount = TIM2->CNT; // Get timer value

        if (measuredCount < SystemCoreClock) {

            Freq = ((float)SystemCoreClock/measuredCount);// Frequency
Calculation

        } else {

            Freq = (measuredCount/(float)SystemCoreClock);// Frequency
Calculation

        }

    }

    // 2. This is the first edge:
    else {

        TIM2->CNT = (uint32_t)0x0;

        /* ENABLE TIM2 COUNTER i.e. start counter because input signal
detected */
        TIM2->CR1 |= TIM_CR1_CEN;
```

```
    }

}

void EXTI0_1_IRQHandler() {

    if ((EXTI->PR & EXTI_PR_PR1) != 0 ) {

        calcFreq();

        // Reset interrupt flag
        EXTI->PR = EXTI_PR_PR1;

    }

    if ((EXTI->PR & EXTI_PR_PR0) != 0 ) {
    /* If inSig = 0, then: let inSig = 1, disable
    EXTI1 interrupts, enable EXTI2 interrupts */
    /* Else: let inSig = 0, disable EXTI2 interrupts,
    enable EXTI1 interrupts */

        inSig = (inSig ^ 0x1);

        if ((inSig & 0x1) == 1) {
            // Mask Line 2, unmask Line 1
            EXTI->IMR &= ~(EXTI_IMR_MR2);
            EXTI->IMR |= EXTI_IMR_MR1;
        } else {
            // Mask Line 1, unmask Line 2
            EXTI->IMR &= ~(EXTI_IMR_MR1);
            EXTI->IMR |= EXTI_IMR_MR2;
        }

        // reset interrupt flag
        EXTI->PR = EXTI_PR_PR0;

    }

}
```

```
/* This handler is declared in system/src/cmsis/vectors_stm32f051x8.c */
void EXTI2_3_IRQHandler() {

    /* Check if EXTI2 interrupt pending flag is indeed set */
    if ((EXTI->PR & EXTI_PR_PR2) != 0) {

        calcFreq();

    }

    EXTI->PR = EXTI_PR_PR2;

}

#pragma GCC diagnostic pop

//
-----------------------------------------------------------------------------
--
```